

# HIV'Report

*TP. Hồ Chí Minh, tháng 12 năm 2025*

# BẢNG PHÂN CÔNG NHIỆM VỤ

| STT | Hạng mục công việc  | Chi tiết                                                                                                                                            | Ghi chú            |
|-----|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| 1   | Architecture Design | <ul style="list-style-type: none"><li>- Thiết kế sơ đồ khối hệ thống</li><li>- Lựa chọn phần cứng (FCU, Companion)</li></ul>                        |                    |
| 2   | Simulation (SITL)   | <ul style="list-style-type: none"><li>- Cài đặt Ubuntu, ROS 2, Gazebo</li><li>- Cấu hình ros_gz_bridge</li><li>- Viết Node điều khiển C++</li></ul> |                    |
| 3   | Hardware Build      | <ul style="list-style-type: none"><li>- In 3D khung vỏ</li><li>- Hàn mạch, đấu nối Motor/ESC</li><li>- Cân chỉnh trọng tâm</li></ul>                |                    |
| 4   | High-level Logic    | <ul style="list-style-type: none"><li>- Path Planning (Lập kế hoạch đường đi)</li><li>- Obstacle Avoidance (Tránh vật cản)</li></ul>                | Tạm thời không cần |

# Mục lục

|          |                                                 |          |
|----------|-------------------------------------------------|----------|
| <b>1</b> | <b>Tổng quan dự án (Project Overview)</b>       | <b>3</b> |
| 1.1      | Lộ trình phát triển (Roadmap) . . . . .         | 3        |
| <b>2</b> | <b>Kiến trúc Hệ thống (System Architecture)</b> | <b>4</b> |
| 2.1      | Kiến trúc Phần cứng . . . . .                   | 4        |
| 2.2      | Nguyên lý Offboard Control . . . . .            | 5        |
| <b>3</b> | <b>Software Stack</b>                           | <b>5</b> |
| 3.1      | Mối quan hệ giữa ROS 2 và Gazebo . . . . .      | 6        |
| <b>4</b> | <b>Lý thuyết Điều khiển (Control Theory)</b>    | <b>7</b> |

# 1 Tổng quan dự án (Project Overview)

Mục tiêu cốt lõi của dự án là xây dựng một hệ thống Drone tự hành hoàn chỉnh, bắt đầu từ việc kiểm chứng giải thuật trong môi trường mô phỏng (Simulation-in-the-Loop - SITL) trước khi triển khai lên phần cứng thực tế. Điều này giúp giảm thiểu rủi ro va chạm và tiết kiệm chi phí linh kiện.

## 1.1 Lộ trình phát triển (Roadmap)

Dự án được chia thành 3 giai đoạn chính:

### 1. Giai đoạn 1: Simulation (Hiện tại)

- Dựng môi trường Gazebo (Physics Engine).
- Thiết kế Node điều khiển bay (Flight Controller) bằng C++ trên ROS 2.
- Cấu hình cầu nối `ros_gz_bridge` để map dữ liệu IMU và Motor.

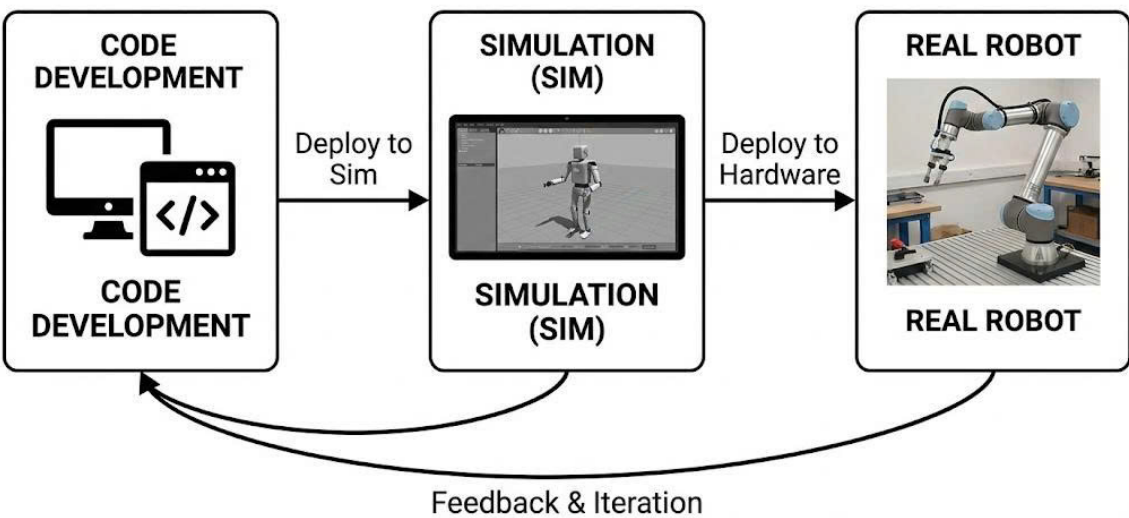
### 2. Giai đoạn 2: Hardware Integration

- Lắp ráp khung máy bay (Airframe).
- Tích hợp máy tính nhúng (Raspberry Pi/Jetson) với Flight Controller (Pixhawk).

### 3. Giai đoạn 3: Real-world Testing

- Bay thử nghiệm (Maiden flight).
- Tinh chỉnh bộ tham số PID (Tuning).

## ROBOT DEVELOPMENT FLOWCHAIR (V-MODEL / AGILE)



Hình 1: Lộ trình phát triển từ Mô phỏng đến Thực tế (V-Model)

---

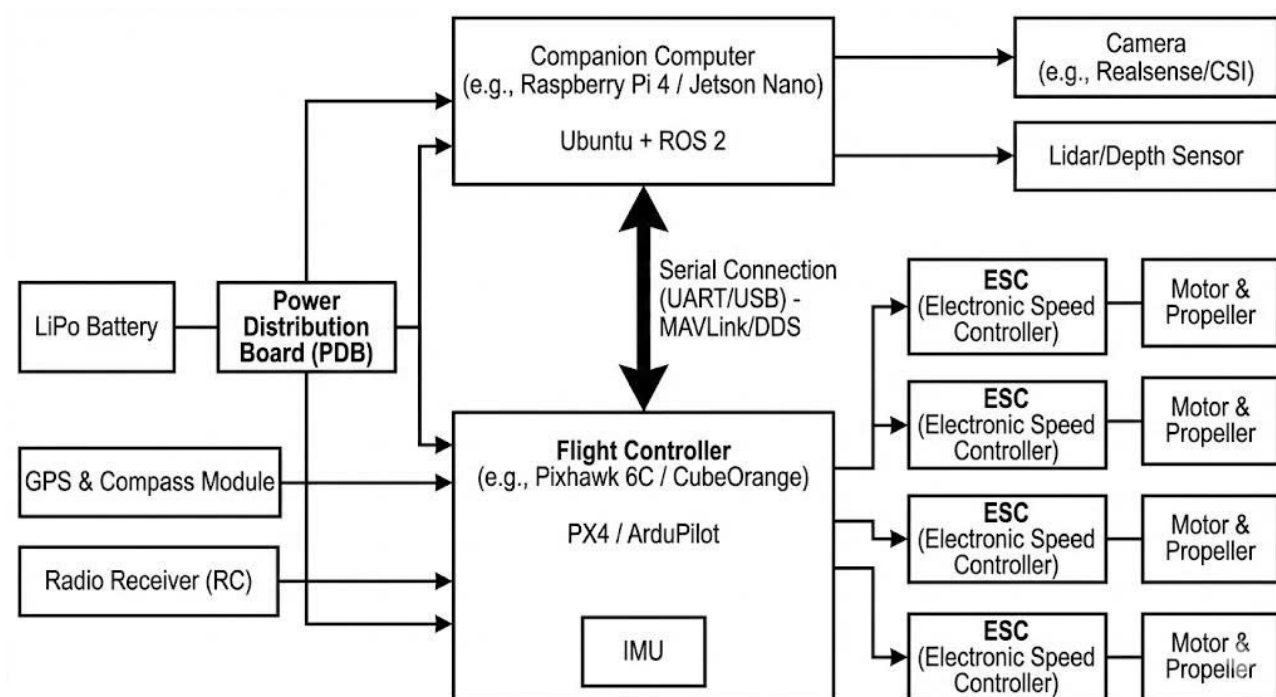
## 2 Kiến trúc Hệ thống (System Architecture)

Hệ thống được thiết kế theo mô hình **Offboard Control**. Đây là kiến trúc hiện đại cho phép tách biệt hoàn toàn giữa các tác vụ sinh tồn (giữ thăng bằng) và các tác vụ thông minh (AI, lập kế hoạch), giúp tăng tối đa tính ổn định và khả năng mở rộng của Robot.

### 2.1 Kiến trúc Phần cứng

Sơ đồ phần cứng của Drone được ví như một cơ thể sinh học, trong đó các thành phần phối hợp chặt chẽ theo phân cấp nhiệm vụ rõ ràng:

- **Low-level Controller (Flight Control Unit - FCU):** Đóng vai trò như "**Tủy sống**" và "**Hệ phản xạ**".
  - **Thiết bị:** Pixhawk 6C hoặc CubeOrange.
  - **Nhiệm vụ:** Chịu trách nhiệm cho các phản xạ vô điều kiện như giữ ổn định máy bay (Attitude Control), đọc cảm biến IMU với tần số rất cao ( $> 500\text{Hz}$ ).
  - **Yêu cầu:** **Hard Real-time** (Thời gian thực cứng) - đảm bảo phản hồi ngay lập tức để ngăn chặn các sự cố vật lý như lật hoặc rơi tự do.
- **High-level Controller (Companion Computer):** Đóng vai trò như "**Bộ não trung ương**".
  - **Thiết bị:** Raspberry Pi 4 (8GB RAM) hoặc Jetson Nano.
  - **Nhiệm vụ:** Thực thi các tác vụ tư duy logic bậc cao trên nền tảng **Ubuntu + ROS 2**. Bao gồm: xử lý ảnh (Computer Vision), chạy thuật toán AI, lập kế hoạch đường đi tối ưu và né vật cản.
  - **Giao tiếp:** Gửi lệnh điều hướng xuống FCU thông qua giao thức MAVLink hoặc Micro-XRCE-DDS (cổng UART/USB).
- **Hệ thống thực thi (Actuators):** Đóng vai trò là "**Cơ bắp**". Bao gồm các bộ điều tốc (ESC) và Motor, chuyển đổi lệnh điện tử thành lực nâng vật lý thông qua sức gió.



Hình 2: Sơ đồ kiến trúc phần cứng: Kết nối giữa Companion Computer và Flight Controller

## 2.2 Nguyên lý Offboard Control

Trong mô hình này, máy tính ROS 2 đóng vai trò chủ động ra lệnh. FCU sẽ nhường quyền điều khiển cho máy tính bên ngoài để thực hiện các quỹ đạo bay phức tạp mà bản thân FCU không tự tính toán được. Cơ chế này giúp bảo vệ hệ thống: nếu bộ não (ROS 2) gặp sự cố, tủy sống (FCU) sẽ tự động kích hoạt chế độ an toàn để bảo vệ Drone.

## 3 Software Stack

| Thành phần   | Công nghệ / Phiên bản                                 |
|--------------|-------------------------------------------------------|
| Hệ điều hành | Ubuntu 22.04 LTS (Jammy Jellyfish)                    |
| Middleware   | <b>ROS 2 Humble Hawksbill</b> (Standard for Robotics) |
| Simulation   | <b>Gazebo Harmonic</b> (Phiên bản mới nhất)           |
| Bridge       | <code>ros_gz_bridge</code> (Cầu nối ROS 2 ↔ Gazebo)   |
| Language     | C++ (cho Control Node), Python (cho AI/Vision)        |

Bảng 1: Danh sách các phần mềm sử dụng

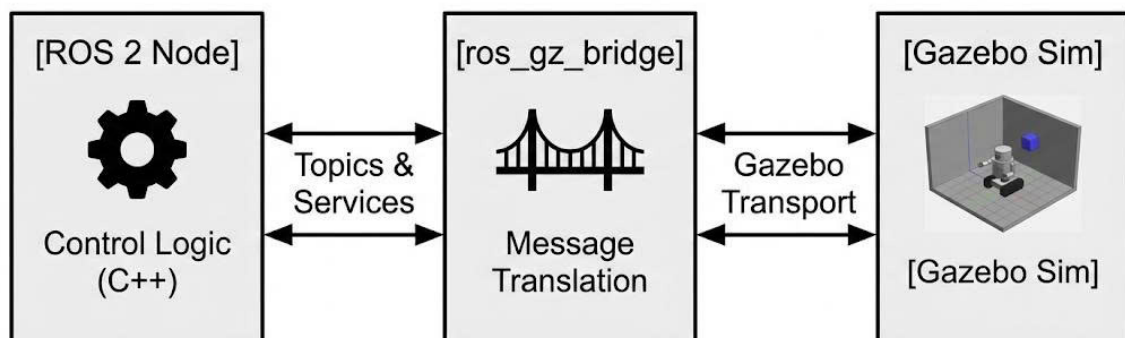
### 3.1 Môi quan hệ giữa ROS 2 và Gazebo

Vì Gazebo và ROS 2 sử dụng cơ chế truyền tin khác nhau (Gazebo Transport vs DDS), chúng ta cần một node trung gian là `ros_gz_bridge`.

*\* Node là gì? \**

Trong kiến trúc ROS 2, một **Node** được hiểu là một thực thể (hoặc một tiến trình) thực hiện một nhiệm vụ chuyên biệt trong hệ thống Robot.

- **Tính độc lập:** Mỗi Node đảm nhận một chức năng duy nhất (ví dụ: Node xử lý camera, Node điều khiển PID, Node đọc cảm biến). Điều này giúp hệ thống hoạt động ổn định, nếu một Node bị lỗi, các thành phần khác vẫn hoạt động bình thường.
- **Cơ chế giao tiếp:** Các Node không đứng cô lập mà kết nối với nhau thông qua mạng lưới các "đường ống" dữ liệu (Topics, Services).
- **Vai trò của `ros_gz_bridge`:** Đây là một Node đặc biệt đóng vai trò như một "người phiên dịch". Nó đứng giữa để chuyển đổi ngôn ngữ dữ liệu của môi trường mô phỏng Gazebo sang ngôn ngữ mà ROS 2 có thể hiểu được và ngược lại.



Hình 3: Sơ đồ kiến trúc giao tiếp dữ liệu giữa ROS 2 và Gazebo thông qua `ros_gz_bridge`

## 4 Lý thuyết Điều khiển (Control Theory)

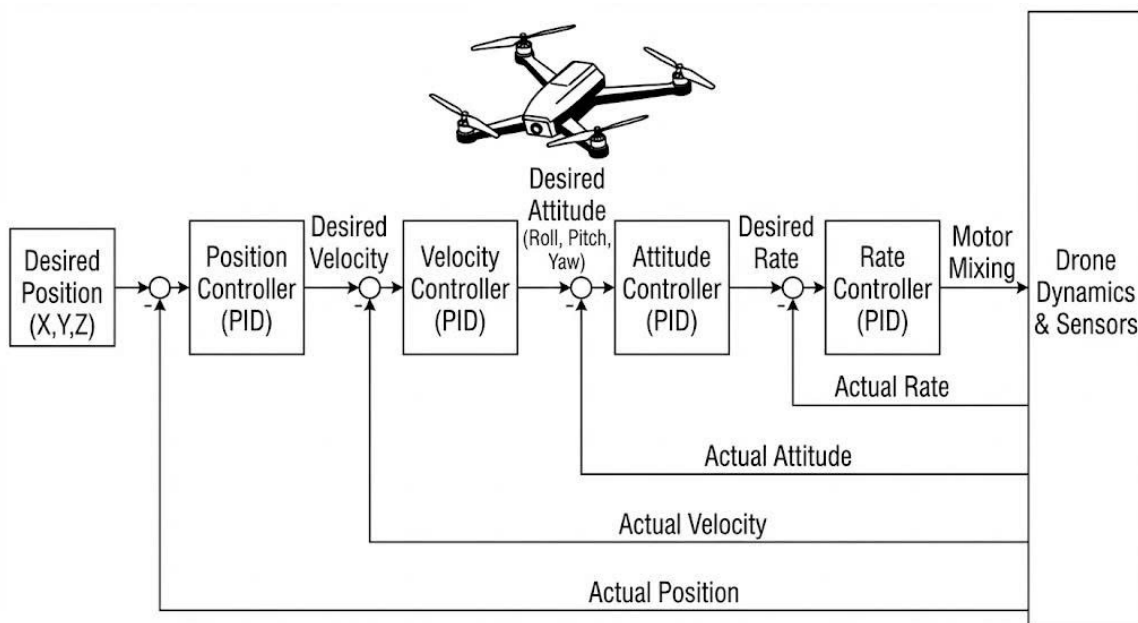
Để Drone bay ổn định, hệ thống điều khiển được thiết kế theo cấu trúc **Vòng lặp xếp chồng (Cascaded Control Loops)**. Node C++ của chúng ta sẽ thực thi các vòng lặp này.

Position → Velocity → **Attitude (Góc)** → **Rate (Vận tốc góc)** → Motor Mixing

Trong đó, phương trình PID cơ bản cho một trục (ví dụ trục Roll) được mô tả như sau:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (1)$$

- $e(t)$ : Sai số giữa góc mong muốn và góc thực tế (lấy từ IMU).
- $u(t)$ : Tín hiệu điều khiển gửi xuống Motor.



Hình 4: Sơ đồ khối hệ thống điều khiển PID xếp chồng (Cascaded Control Loops)

 **Video tham khảo:** *Drone Simulation and Control, Part 1: Setting Up the Control Problem*